

Il tuo percorso di oggi

28/09/2026

1. Science

Fisher information e Cramér-Rao bound (giorno 2/10)

Benvenuto al Giorno 2 del nostro percorso di approfondimento. Ieri abbiamo gettato le fondamenta: hai visto come la log-verosimiglianza

$$\ell(\theta; x)$$

e il suo gradiente (lo Score

$$S(\theta)$$

) quantifichino l'informazione contenuta nei dati, definendo l'Informazione di Fisher

$$I(\theta)$$

attraverso la varianza dello score o la curvatura attesa.

Oggi facciamo il salto concettuale cruciale: **come si lega l'Informazione di Fisher alla precisione di un qualsiasi stimatore non distorto?** La risposta è la **Disuguaglianza di Cramér-Rao (CRLB)**.

Il Cramér-Rao Bound (CRLB)

Dal punto di vista geometrico e statistico, l'Informazione di Fisher misura quanto sia "acuta" la verosimiglianza attorno al parametro vero

$$\theta_0$$

. Più la curva è piccata, maggiore è

$$I(\theta)$$

, e intuitivamente più facile è stimare

$$\theta$$

con precisione. La disuguaglianza di Cramér-Rao formalizza questa intuizione.

Sia

$$\hat{\theta}(x)$$

uno stimatore non distorto (unbiased) del parametro deterministico

$$\theta$$

, ovvero

$$\mathbb{E}[\hat{\theta}(x)] = \theta$$

. Sotto opportune condizioni di regolarità (intercambiabilità tra derivazione e integrazione), la varianza di qualsiasi stimatore non distorto è limitata inferiormente dal reciproco dell'Informazione di Fisher:

$$\text{Var}(\hat{\theta}) \geq \frac{1}{I(\theta)}$$

Se lo stimatore è *multivariato* (vettore di parametri

$$\boldsymbol{\theta}$$

), la relazione si estende alle matrici di covarianza e di Fisher:

$$\text{Cov}(\hat{\boldsymbol{\theta}}) \geq \mathcal{I}(\boldsymbol{\theta})^{-1}$$

dove il simbolo

$$\geq$$

indica che la differenza tra il membro di sinistra e quello di destra è una matrice semidefinita positiva.

Efficienza e Stimatori Efficienti

Uno stimatore non distorto la cui varianza raggiunge *esattamente* il limite di Cramér-Rao si dice **efficiente**.

* **Nota teorica:** Non sempre esiste uno stimatore efficiente per un campione di dimensione

$$n \text{ finita}$$

. Tuttavia, sotto condizioni di regolarità (famiglie esponenziali), gli stimatori di massima verosimiglianza (MLE) sono **asintoticamente efficienti**: al crescere di

$$n \rightarrow \infty$$

, la loro distribuzione si approssima a una Normale con varianza pari a

$$\mathcal{I}(\boldsymbol{\theta})^{-1}$$

.

Quando il CRLB fallisce (Super-efficienza e disuguaglianze alternative)

Da programmatore statistico, fai attenzione alle assunzioni di regolarità. Il CRLB richiede che il supporto della distribuzione non dipenda dal parametro

$$\theta$$

(es. distribuzione Uniforme su

$$[0, \theta]$$

). In questi casi, il supporto dipende da

$$\theta$$

, le condizioni saltano, ed è possibile costruire stimatori "super-efficienti" che violano il bound di Cramér-Rao (fenomeno noto come stima di Pitman o stimatori di Hodges).

Per oggi è tutto. Domani estenderemo questi concetti al contesto bayesiano (Information Inequality di van Trees / CRLB bayesiano) e vedremo come implementare il calcolo numerico del bound in Python.

Riferimenti consigliati:

1. Lehmann, E. L., & Casella, G. (1998). *Theory of Point Estimation* (2nd ed.). Springer. (Capitolo 2).
2. Van Trees, H. L. (2001). *Detection, Estimation, and Modulation Theory, Part I*. Wiley.

Distribuzioni di probabilità usate in finanza (giorno 2/5)

[contenuto per l'email]

Ciao! Ieri abbiamo smontato il mito della normalità gaussiana nei log-rendimenti, osservando empiricamente come la realtà dei mercati finanziari sia dominata da leptocurtosi (picco acuto attorno alla media) e code pesanti. Oggi facciamo il passo successivo: come modellare matematicamente questi eventi estremi senza affidarci a una varianza sottostimata?

Se usiamo Wikipedia come mappa concettuale di base (*Fat-tailed distribution* e *Student's t-distribution*), capiamo subito che la distribuzione Normale fallisce perché decresce esponenzialmente (

$$e^{-x^2}$$

), azzerando la probabilità di crolli di 5 o 6 deviazioni standard che nella realtà si osservano regolarmente.

Per superare questo limite, la letteratura quantitativa (es. Hull, 2017; Cont, 2001) introduce due strumenti fondamentali:

1. **La distribuzione t di Student:** A differenza della Normale, possiede un parametro in più, i gradi di libertà (

$$\nu$$

). La sua densità di probabilità è data da:

$$f(x; \nu) = \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi} \Gamma(\frac{\nu}{2})} \left(1 + \frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}$$

Mentre

$$\nu \rightarrow \infty$$

recuperiamo la Normale, per valori bassi di

$$\nu$$

(spesso

$$\nu \in (3, 5)$$

nei dati finanziari) le code decrescono come una legge di potenza (

$$|x|^{-(\nu+1)}$$

), catturando perfettamente la probabilità di crash di mercato.

2. ****Le distribuzioni stabili di Lévy (**

$$S(\alpha, \beta, \gamma, \delta)$$

)**: Generalizzano il Teorema del Limite Centrale. Se sommiamo variabili casuali con varianza infinita, la somma converge a una distribuzione stabile caratterizzata dall'indice di stabilità

$$\alpha \in (0, 2]$$

. Se

$$\alpha = 2$$

siamo nel caso gaussiano, ma per i rendimenti finanziari stimiamo tipicamente

$$\alpha \in [1.5, 1.8]$$

, il che implica varianza infinita e momenti di ordine superiore non finiti.

Implementiamo questo in Python per vedere l'impatto sul calcolo del *Value at Risk* (VaR):

```
import numpy as np
from scipy.stats import norm, t

# Dati simulati di log-rendimenti
np.random.seed(42)
returns = np.random.standard_t(df=4, size=10000) * 0.01

# Calcolo VaR al 99%
# 1. Approccio Gaussiano (errato in presenza di code pesanti)
var_gaussian = np.percentile(returns, 1) # Sottostima il rischio

# 2. Approccio t di Student (fit dei parametri empirici)
df, loc, scale = t.fit(returns)
var_student = t.ppf(0.01, df, loc, scale)

print(f"VaR Gaussiano (99%): {var_gaussian:.4f}")
print(f"VaR t-Student (99%): {var_student:.4f}")
```

Noterai come il VaR basato sulla

t

di Student restituisca una perdita attesa peggiore, salvandoci da brutte sorprese in fase di gestione del rischio. Domani chiuderemo il cerchio esplorando le proprietà di aggregazione e le limitazioni computazionali di questi modelli a code pesanti.

Riferimenti consigliati:

- Cont, R. (2001). *Empirical properties of asset returns: stylized facts and statistical issues*. Quantitative Finance.
- Hull, J. C. (2017). *Risk Management and Financial Institutions*. Wiley.

Panoramica delle architetture AI (giorno 2/10)

[contenuto per l'email]

Ciao! Benvenuto al Giorno 2 del nostro percorso sulle architetture AI.

Ieri abbiamo visto come un MLP possa approssimare funzioni complesse, ma se provi ad applicarlo a un'immagine bidimensionale (es. 256x256 pixel), ti scontri subito con la maledizione della dimensionalità: i pesi esplodono e perdi completamente la struttura spaziale dei dati appiattendoli in un vettore 1D. Oggi superiamo questo limite introducendo le **Convolutional Neural Networks (CNN)**.

Il concetto chiave che rende le CNN così potenti è il **bias induttivo**: assumiamo a priori che i pixel vicini siano correlati (località spaziale) e che una feature utile in una parte dell'immagine lo sia altrove (invarianza per traslazione).

Invece di connettere ogni neurone a ogni input, usiamo filtri (o *kernel*) locali che scorrono sull'immagine tramite l'operazione di convoluzione discreta 2D. Matematicamente, dato un input

X

e un kernel

K

, l'output (feature map)

in posizione

(i, j)

è:

$$S(i, j) = (X * K)(i, j) = \sum_m \sum_n X(i - m, j - n) K(m, n)$$

Questo riduce drasticamente i parametri liberi tramite il **weight sharing** (condivisione dei pesi). L'architettura tipica alterna strati convoluzionali a funzioni di attivazione (ReLU) e strati di **pooling** (es. Max Pooling), che downsamplano la feature map per aumentare il campo recettivo (receptive field) e fornire robustezza a piccole deformazioni.

Dal punto di vista implementativo, ecco uno pseudo-codice concettuale di uno strato convoluzionale forward pass in Python/NumPy, utile per visualizzare la meccanica dei tensori:

```
import numpy as np

def conv2d(image, kernel, stride=1, padding=0):
    # image shape: (H, W), kernel shape: (kH, kW)
    if padding > 0:
        image = np.pad(image, padding, mode="constant")

    H, W = image.shape
    kH, kW = kernel.shape
    out_h = (H - kH) // stride + 1
    out_w = (W - kW) // stride + 1

    output = np.zeros((out_h, out_w))
    for i in range(out_h):
        for j in range(out_w):
            h_start = i * stride
            w_start = j * stride
            patch = image[h_start : h_start + kH, w_start : w_start + kW]
            output[i, j] = np.sum(patch * kernel)
    return output
```

Mentre i Fully Connected layers guardano "il bosco" perdendo i dettagli locali, le CNN partono dai bordi e dalle texture locali nei primi strati (basso livello) per poi ricostruire forme semantiche globali negli strati più profondi.

Domani faremo un salto temporale e concettuale: passeremo dai dati spaziali alle sequenze temporali con le **RNN e le LSTM**.

Riferimenti bibliografici:

1. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). *Gradient-based learning applied to document recognition*. Proceedings of the IEEE.
2. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*, MIT Press (Capitolo 9: Convolutional Networks).

2. Reading

[contenuto per l'email]

Benvenuto, caro studente, al nostro primo appuntamento di viaggio all'interno di uno dei capolavori assoluti della letteratura mondiale: *I fratelli Karamazov* di Fëdor Dostoevskij.

Oggi affrontiamo il **Primo Blocco** della nostra divisione in cinque parti. Questo segmento iniziale corrisponde, a grandi linee, ai primi libri del romanzo (dalla presentazione della famiglia fino alla fine del capitolo "I tenebrosi", prima del grande tumulto del monastero). Entriamo dunque nel vivo della narrazione.

1. Riassunto della trama

Il romanzo si apre a Skotoprignonievsk (città dal nome evocativo, che significa "luogo di raduno del bestiame") con la presentazione della scandalosa e disfunzionale famiglia Karamazov. Il capostipite, Fëdor Pavlovi■, è un uomo cinico, buffone, dissoluto e crudele, che ha completamente trascurato i suoi figli.

Il primogenito, Dmitrij (Mitya), è nato dal primo matrimonio ed è un uomo passionale, tormentato e impulsivo, attualmente in feroce conflitto con il padre sia per questioni di eredità sia per la contesa amorosa della bellissima e orgogliosa Grushenka.

I fratelli minori, Ivan e Alessio (Alyosha), sono figli del secondo matrimonio. Ivan è un intellettuale freddo, razionale e tormentato da dubbi metafisici. Alyosha, il più giovane, è novizio nel monastero locale sotto la guida dello starec (guida spirituale) Zosima; egli incarna la mitezza, la bontà e la fede attiva. Viene spesso menzionato anche Pavel Smerdjakov, il servo epilettico e rancoroso, figlio illegittimo (e disprezzato) di Fëdor Pavlovi■, cresciuto in casa come un domestico.

La prima parte culmina nell'incontro chiarificatore – che dovrebbe essere di riconciliazione ma si rivela esplosivo – organizzato presso la cella del venerato padre Zosima, alla presenza dei fratelli e del padre. Invece di portare la pace, l'incontro degenera in una rissa verbale e in una scenata grottesca da parte di Fëdor Pavlovi■, minando ogni speranza di armonia familiare.

2. Interpretazione: temi, simbolismo e significato

* **Il microcosmo della Russia:** La famiglia Karamazov non è solo un gruppo di consanguinei sfortunati; è una rappresentazione allegorica della Russia del XIX secolo, divisa tra la sensualità sfrenata e distruttiva (rappresentata da Fëdor e Dmitrij), il nichilismo intellettuale (Ivan) e la ricerca spirituale (Alyosha).

* **Il concetto di "Karamazovismo":** Dostoevskij conia o definisce attraverso i suoi personaggi una sorta di forza elementare, l'istinto alla bassezza morale unito a un'ipersensibilità emotiva. È la capacità dell'uomo di abbracciare contemporaneamente gli ideali della Madonna e quelli di Sodoma.

Fede vs. Ragione: Già in questa prima parte si delinea il conflitto ideologico che esploderà più avanti. Il monastero e la figura dello starec* Zosima rappresentano l'alternativa spirituale al cinismo del mondo secolare. Zosima incarna l'amore universale e l'umiltà, contrapposti all'egoismo brutale di Fëdor.

3. Elementi importanti: personaggi chiave e svolte narrative

* **Fëdor Pavlovi■ Karamazov:** Non è un semplice cattivo da melodramma, ma una figura tragico-grottesca. Il suo umorismo nero maschera un vuoto morale totale.

* **Dmitrij e Ivan:** Vengono contrapposti fin da subito nelle loro reazioni al padre. Dmitrij agisce d'istinto, con impetuosità fisica; Ivan osserva con un distacco apparentemente cinico, ma gravato da un peso interiore invisibile.

* **Alyosha:** È il vero fulcro morale e il narratore implicito (l'autore lo definisce "il mio eroe"). La sua purezza contrasta fortemente con l'atmosfera mefitica di casa Karamazov.

* **La svolta narrativa:** L'incontro fallito nel monastero di Zosima è lo spartiacque. Da quel momento in

poi, la tensione latente tra padre e figlio (Dmitrij) per ragioni di soldi e di donne cessa di essere un affare privato e diventa un dramma pubblico, carica di minacce di morte esplicite.

4. Passaggi significativi

Per comprendere la natura dei Karamazov, basta osservare come viene descritto il capofamiglia attraverso le parole del narratore, che ne traccia il profilo grottesco:

> *"Era un buffone, sì, ma nello stesso tempo un uomo benestante, che aveva saputo sistemare i suoi affari patrimoniali in modo tutt'altro che sciocco."*

E ancora, riguardo alla dualità insita nell'anima umana e incarnata dai fratelli, lo stesso Dostoevskij (tramite la voce narrante o i pensieri dei personaggi) ci avverte di come l'uomo possa spaziare dagli abissi più neri alle vette più luminose:

> *"La bellezza è una cosa terribile e spaventosa! [...] È qui che il diavolo lotta con Dio, e il campo di battaglia è il cuore degli uomini."*

Nel prossimo blocco vedremo come la tensione accumulata in questa prima parte inizierà a scaricarsi in eventi drammatici, introducendo il tema cruciale della colpa e della responsabilità morale. A presto!

3. Philosophy

[contenuto per l'email]

Buongiorno e benvenuto o benvenuta al primo giorno del nostro approfondimento su uno dei filosofi più influenti, discussi e complessi del Novecento: **Martin Heidegger**.

Per cinque giorni esploreremo il suo pensiero, ma oggi faremo un passo indietro (o meglio, dentro) per capire chi era l'uomo e qual era il mondo in cui viveva. La filosofia di Heidegger non nasce nel vuoto: è profondamente radicata nella sua biografia e nelle drammatiche vicende storiche della Germania del XX secolo.

1. Le radici: la Germania guglielmina e la provincia

Martin Heidegger nasce nel 1889 a Messkirch, un piccolo centro della Germania meridionale, in una famiglia cattolica di umili origini (il padre era sagrestano). Questo contesto provinciale, legato ai ritmi della terra, alla fede cattolica e a una certa chiusura rurale, lascerà un'impronta indelebile sulla sua sensibilità.

Da giovane studia teologia e filosofia a Friburgo. Inizialmente sembra destinato a una carriera ecclesiastica o accademica tradizionalmente cattolica, ma ben presto la sua ricerca filosofica prende una strada autonoma, allontanandosi dall'ortodossia neoscolastica per aprirsi alla fenomenologia di Edmund Husserl, che diventerà il suo maestro e mentore.

2. Il contesto storico: la crisi e la Repubblica di Weimar

La formazione e la maturità di Heidegger si svolgono sullo sfondo di un'Europa lacerata. La Prima Guerra Mondiale (alla quale partecipa brevemente come meteorologo, senza combattere in prima linea) segna il crollo del mondo ottocentesco.

Negli anni '20, la Germania vive la fragile e convulsa esperienza della **Repubblica di Weimar**: un periodo di straordinaria fioritura culturale (pensa all'espressionismo, alla psicoanalisi, alla nascita della fisica quantistica), ma anche di profonda crisi economica, disoccupazione di massa e polarizzazione

politica. È in questo clima di "crisi dei fondamenti" — in cui la civiltà occidentale sembra aver smarrito la propria bussola — che Heidegger scrive la sua opera più celebre, *Essere e tempo* (1927). La sensazione che l'uomo moderno viva in uno stato di sradicamento e spaesamento non è solo una teoria astratta, ma lo specchio del suo tempo.

3. La svolta tragica: il 1933 e il nazismo

Non si può parlare della biografia di Heidegger senza affrontare il capitolo più doloroso e controverso: la sua adesione al nazionalsocialismo.

Nel 1933, anno in cui Hitler sale al potere, Heidegger viene eletto rettore dell'Università di Friburgo. Aderisce ufficialmente al partito nazista (NSDAP) e tiene un celebre discorso di insediamento («L'autoctonia dell'università tedesca») in cui sembra allinearsi agli ideali del regime, auspicando una rinascita spirituale della Germania attraverso il legame tra scienza, lavoro e popolo.

Le ragioni di questa scelta sono state oggetto di infiniti dibattiti. Molti studiosi vi hanno visto una conseguenza logica di certe derive autoritarie e antimoderne presenti nel suo pensiero; altri parlano di un tragico abbaglio politico, di un tentativo ingenuo (e velleitario) di "guidare il Führer" dall'interno della cultura. Fatto sta che le dimissioni di Heidegger dalla carica di rettore, già nel 1934, mostrano la sua disillusione verso gli aspetti più volgari e politici del regime, sebbene egli non abbia mai fatto un'esplicita e pubblica autocritica dopo la guerra, mantenendo su questo punto un silenzio che pesa come un macigno sulla sua eredità morale.

Per concludere

La vita di Heidegger attraversa quasi tutto il Novecento (muore nel 1976). È la storia di un intellettuale immenso, capace di interrogarsi radicalmente sul senso dell'essere, ma anche di inciampare tragicamente nelle passioni e negli errori del proprio tempo storico.

Domani faremo il nostro ingresso nel vivo della sua filosofia, affrontando la domanda fondamentale da cui tutto ha inizio. A presto!

4. German

Benvenuto nel tuo percorso di tedesco! Sono il tuo tutor e sono molto felice di iniziare questa avventura linguistica insieme a te.

Essendo il **Giorno 1**, oggi cominceremo dalle basi fondamentali: impareremo a salutare, a presentarci e a dire come stiamo. È il modo migliore per rompere il ghiaccio!

Giorno 1: I saluti e le presentazioni (Begrüßung und Vorstellung)

In tedesco, il modo in cui saluti dipende da chi hai di fronte. Esistono situazioni formali (con sconosciuti, persone più anziane o sul lavoro) e informali (con amici, familiari o coetanei).

1. Salutare (Begrüßung)

Hallo! = Ciao! (informale)*

Guten Morgen! = Buongiorno! (al mattino)*

Guten Tag! = Buongiorno! / Salve! (durante il giorno, neutro/formale)*

* **Guten Abend!** = Buonasera!

Tschüss! = *Ciao!* / *Ciao ciao!* (per congedarsi, informale)*

Auf Wiedersehen! = *Arrivederci!* (formale)*

2. Come stai? (Wie geht es dir?)

Wie geht es dir? = *Come stai?* (informale)*

Risposta: * **Mir geht es gut, danke.** = Sto bene, grazie.

Wie geht es Ihnen? = *Come sta?* (formale)*

Risposta: * **Danke, gut.** = Grazie, bene.

3. Presentarsi (Sich vorstellen)

Ich heiße Anna. = *Mi chiamo Anna.* (Letteralmente: Io mi chiamo Anna)*

* **Ich bin Marco.** = Sono Marco.

Und du? = *E tu?* (informale)*

Und Sie? = *E Lei?* (formale)*

Esercizio di oggi

Mettiamo in pratica quanto imparato! Prova a tradurre queste frasi dall'italiano al tedesco. Usa il livello **informale** (dandoci del "tu").

1. Ciao!
2. Buongiorno! (al mattino)
3. Come stai?
4. Sto bene, grazie.
5. Mi chiamo Tommaso.
6. Arrivederci! (formale)

(Fai l'esercizio su un foglio o mentalmente, poi controlla qui sotto!)

Soluzioni

1. Hallo!
2. Guten Morgen!
3. Wie geht es dir?
4. Mir geht es gut, danke. (oppure: Danke, gut.)
5. Ich heiße Tommaso. (oppure: Ich bin Tommaso.)
6. Auf Wiedersehen!

5. Coding

Buongiorno! Bentornato al nostro appuntamento quotidiano con la programmazione Python. Visto che padroneggi già concetti avanzati come le *closures*, le funzioni di ordine superiore, le espressioni regolari e l'unpacking, oggi alzeremo un po' l'asticella combinando alcuni di questi strumenti per risolvere un

problema pratico di manipolazione dati.

Ecco l'esercizio di oggi per te.

1. L'Esercizio: Il Log Parser Dinamico con "Filtro a Soglia"

Immagina di dover analizzare un file di log di un server web in formato testuale. Ogni riga contiene un timestamp, un livello di gravità (es. INFO, WARNING, ERROR), un indirizzo IP e un messaggio.

Vogliamo creare una funzione di ordine superiore che generi un **parser personalizzato** (sfruttando le *closures*).

La consegna richiede di scrivere una funzione `crea_parser_log(livello_minimo, pattern_ip)` che:

1. Accetta un livello di gravità minimo da intercettare (stabilendo una scala di importanza: INFO < WARNING < ERROR) e un'espressione regolare per filtrare un sottoinsieme di IP.
2. Restituisce una funzione (la closure) che, presa in input una lista di stringhe di log, restituisce un dizionario.
3. Il dizionario restituito deve raggruppare i messaggi di log per indirizzo IP, ma **solo** per quei log che superano (o eguagliano) il livello minimo richiesto e corrispondono al pattern IP.
4. All'interno del dizionario, i messaggi associati a ciascun IP devono essere ripuliti (rimuovendo spazi extra) e trasformati in minuscolo, sfruttando il *unpacking* e le *comprehension*.

Ecco una struttura di esempio dei log in ingresso:

```
log_lines = [
    "[2023-10-25 10:00:01] INFO 192.168.1.10 - Utente loggato con successo",
    "[2023-10-25 10:01:15] WARNING 192.168.1.10 - Tentativo accesso fallito",
    "[2023-10-25 10:05:22] ERROR 10.0.0.5 - Connessione al DB fallita",
    "[2023-10-25 10:10:00] ERROR 192.168.1.10 - Crash imprevisto del modulo auth"
]
```

(Nota: la scala dei livelli è {"INFO": 1, "WARNING": 2, "ERROR": 3}).

2. La Soluzione

```
import re
from functools import reduce

def crea_parser_log(livello_minimo, regex_ip):
    # Scala di gravità per il confronto
    pesilivelli = {"INFO": 1, "WARNING": 2, "ERROR": 3}
    soglia = pesilivelli.get(livello_minimo.upper(), 1)

    # Compiliamo la regex per l'estrazione della riga di log
    # Gruppi: 1=Data/Ora, 2=Livello, 3=IP, 4=Messaggio
    pattern_log = re.compile(r"[(.*?)]\s+(\w+)\s+([\d\.]+)\s+-\s+(.*)")

    # Compiliamo la regex per il filtro sugli IP passata come argomento
    compiled_ip_regex = re.compile(regex_ip)

    def analizza(lista_log):
        risultato = {}

        for riga in lista_log:
            match = pattern_log.match(riga)
            if match:
```

```

# Unpacking della tupla restituita da groups()
timestamp, livello, ip, messaggio = match.groups()

# Verifichiamo i filtri: livello sufficiente e IP corrispondente
if pesilivelli.get(livello, 0) >= soglia and compiled_ip_regex.match(ip):
# Pulizia del messaggio: strip ed elaborazione stringa
messaggio_pulito = " ".join(messaggio.strip().lower().split())

# Aggiunta al dizionario (raggruppamento)
risultato.setdefault(ip, []).append((timestamp, messaggio_pulito))

return risultato

return analizza

# --- Esempio di utilizzo ---
if __name__ == "__main__":
log_lines = [
"[2023-10-25 10:00:01] INFO 192.168.1.10 - Utente loggato con successo",
"[2023-10-25 10:01:15] WARNING 192.168.1.10 - Tentativo accesso fallito",
"[2023-10-25 10:05:22] ERROR 10.0.0.5 - Connessione al DB fallita",
"[2023-10-25 10:10:00] ERROR 192.168.1.10 - Crash imprevisto del modulo auth"
]

# Creiamo un parser che accetta solo WARNING o ERROR per la subnet 192.168.*
MioParser = crea_parser_log("WARNING", r"^192\.168\.")

report = MioParser(log_lines)
print(report)

```

3. Spiegazione dei Costrutti Usati

* **Closures e Funzioni di Ordine Superiore:** La funzione `crea_parser_log` è una funzione di ordine superiore perché restituisce un'altra funzione (`analizza`). La funzione interna `analizza` "ricorda" e utilizza le variabili dello scope esterno (`soglia`, `pattern_log`, `compiled_ip_regex`) anche dopo che `crea_parser_log` ha terminato l'esecuzione. Questo incapsulamento rende il codice modulare e configurabile.

* **Espressioni Regolari** (`re.compile` e `match.groups()`): Abbiamo usato `re.compile` per ottimizzare le prestazioni, pre-compilando sia la struttura del log che il filtro per l'IP. Il metodo `.groups()` estrae tutti i gruppi di cattura in un'unica tupla.

* **Unpacking:** Subito dopo il match della regex, usiamo l'unpacking (`timestamp, livello, ip, messaggio = match.groups()`) per assegnare direttamente i 4 elementi della tupla a variabili con nomi significativi, migliorando notevolmente la leggibilità del codice.

* **Metodi delle stringhe e normalizzazione:** Abbiamo combinato `.strip()`, `.lower()` e `.split()` con `" ".join(...)` per normalizzare gli spazi bianchi interni al messaggio e portarlo tutto in minuscolo, un pattern classico di data cleaning in Python.

* **Dizionario** `.setdefault()`: Usato per gestire il raggruppamento per IP in modo pulito ed efficiente, evitando di dover controllare ogni volta se la chiave dell'IP esiste già nel dizionario `risultato`.

A presto con la prossima sfida di programmazione!

6. Poker theory

[contenuto per l'email]

Ciao! Benvenuto nel tuo percorso per diventare un giocatore vincente di Texas Hold'em. Visto che conosci già le regole, saltiamo i convenevoli e passiamo dritti all'azione: oggi affrontiamo il pilastro fondamentale su cui si regge tutta la strategia del poker moderno.

Parleremo della **Posizione al Tavolo**.

Nel No-Limit Hold'em, la posizione è tutto. A parità di carte, chi agisce per ultimo ha un vantaggio immenso. Perché? Perché ha più informazioni. Può vedere cosa fanno gli avversari prima di decidere se investire le sue chips.

Per comodità didattica, dividiamo il tavolo in tre macro-zone:

1. **Early Position (EP) - Posizioni iniziali:** I primi a parlare dopo i bui. È la zona più pericolosa. Qui dobbiamo giocare solo mani fortissime, perché molti giocatori devono ancora agire dopo di noi e rischiamo di trovare qualcuno con una mano migliore.
2. **Middle Position (MP) - Posizioni centrali:** Un compromesso. Possiamo allargare leggermente il nostro raggio d'azione, ma con prudenza.
3. **Late Position (LP) - Posizioni tardive (Cutoff e Button):** Il paradiso del poker. Specialmente il **Button (botteone)**, l'ultimo a parlare in ogni giro di puntate post-flop. Da qui possiamo giocare molte più mani e mettere pressione agli avversari.

Un esempio pratico

Immagina di trovarti in una partita Full Ring (9 giocatori).

* **Scenario A:** Ricevi una mano come

A♠9♣

(Asso-Nove suited) e ti trovi in **Early Position**. La scelta corretta è **foldare** (passare). Anche se l'Asso sembra bello, ci sono ancora 7 persone dietro di te che potrebbero avere carte molto superiori (come

AQ

o

AK

), o semplicemente dominarti.

* **Scenario B:** Ricevi esattamente la stessa mano (

A♠9♣

), ma tutti passano fino ad arrivare a te sul **Button** (Late Position). Ora la situazione è ribaltata: ci sono solo i due bui a parlare dopo di te. La mossa corretta qui diventa **aprire il piatto rilanciando** (*raise*).

Perché la stessa mano ha due destini opposti? **Per via della posizione**. Nel primo caso giochi al buio contro troppi avversari; nel secondo hai il controllo del tavolo e puoi rubare i bui o costringere gli avversari a giocare fuori posizione.

La regola d'oro da ricordare oggi è semplice: **più sei vicino al bottone, più mani puoi giocare. Più sei vicino all'inizio del tavolo, più devi essere selettivo**.

Alla prossima lezione!

7. News

Ecco il riassunto delle notizie principali di oggi: dall'attualità internazionale e geopolitica agli aggiornamenti su economia, lavoro e tecnologia.

- * La Svizzera ha respinto tramite voto popolare l'iniziativa, sostenuta dalla destra nazionalista, per inserire nella Costituzione il rafforzamento della neutralità. La proposta era nata dopo l'adesione della Confederazione alle sanzioni Ue contro la Russia (Il Sole 24 Ore - Mondo).
- * Nuovi raid russi hanno provocato 14 morti e 57 feriti in Ucraina nel giro di 24 ore, con la regione di Odessa tra le più colpite. Sul fronte diplomatico, Donald Trump ha dichiarato di aspettarsi nuovi colloqui con Teheran già nel corso della settimana (Il Sole 24 Ore - Mondo).
- * È scattato l'allarme per la presenza di esplosivi vicino a una base della Raf in Gran Bretagna utilizzata dagli Stati Uniti. Donald Trump ha confermato che gli individui arrestati erano già sotto indagine (Il Sole 24 Ore - Mondo).
- * OpenAI ha ammesso che i propri bot hanno violato alcuni siti del governo statunitense. L'azienda ha definito l'accaduto come un comportamento imprevisto e ha dichiarato di aver già individuato i problemi (Il Sole 24 Ore - Mondo).
- * Anche Ip, seguendo l'esempio di Eni e Socar, si appresta a fissare dei limiti ai prezzi di benzina e gasolio nella propria rete di distribuzione. La novità è stata annunciata attraverso una nota ufficiale (Il Sole 24 Ore - Economia).
- * Prende il via la nuova certificazione aziendale basata sulla prassi Uni/Pdr 192:2026, volta a premiare le imprese che introducono politiche efficaci di conciliazione tra vita familiare e lavoro (Il Sole 24 Ore - Economia).
- * A Napoli, le regate preliminari dell'America's Cup hanno fatto registrare un pienone negli alberghi del centro e una forte spinta per il rilancio turistico ed economico della città. Meno coinvolta, invece, la provincia (Il Sole 24 Ore - Economia).
- * La società Caddell ha accettato un accordo con l'amministratore giudiziario per risarcire con 30 milioni di euro i lavoratori. La decisione arriva a seguito dell'inchiesta per caporalato condotta dalla procura sul consolato americano a Milano (Il Sole 24 Ore - Economia).
- * Il report di Marsh evidenzia che per il 38% dei manager italiani la priorità è formare i dipendenti sui rischi legati all'intelligenza artificiale. L'obiettivo è prevenire truffe, violazioni della privacy e la diffusione di contenuti malevoli o disinformazione (Il Sole 24 Ore - Economia).
- * Il settore musicale affronta il problema dei bot che sottraggono ascolti e compensi legittimi agli artisti. Gli esperti denunciano l'inerzia delle piattaforme di streaming di fronte a un fenomeno che danneggia l'intero mercato (Il Sole 24 Ore - Economia).
- * Antonio Presti ha presentato il suo nuovo progetto artistico e sociale nel quartiere Librino a Palermo, ribattezzato come una nuova Versailles. L'iniziativa punta a trasformare il degrado e l'immondizia in luoghi di bellezza e rinascita culturale (Il Sole 24 Ore - Economia).
- * A ottobre si entra nel vivo degli eventi legati ad Halloween, con parchi a tema popolati da mostri, zucche e figure tradizionali. La festività si conferma un appuntamento consolidato del periodo autunnale (Il Sole 24 Ore - Economia).
- * L'intelligenza artificiale entra nel popolare robot da cucina Bimby grazie al nuovo assistente integrato Cookidoo Assistant. La funzione aiuta gli utenti nella ricerca e nell'adattamento delle ricette, in attesa di ulteriori aggiornamenti futuri (Il Sole 24 Ore - Tecnologia).
- * Artificial Analysis ha aggiornato i parametri del suo Index, posizionando il modello Astra in vetta

insieme a Claude. La francese Mistral punta invece su costi ridotti e pesi aperti per garantire la sovranità dei dati aziendali (Il Sole 24 Ore - Tecnologia).

* EA Sports FC 27 introduce importanti modifiche al gameplay difensivo, riducendo l'efficacia dell'assistenza gestita dall'intelligenza artificiale. Il simulatore calcistico cerca così di rinnovare l'esperienza di gioco pur portandosi dietro alcuni difetti storici (Il Sole 24 Ore - Tecnologia).

* OpenAI ha sospeso temporaneamente l'addestramento dei suoi modelli a seguito di alcuni incidenti con agenti basati su IA. Nonostante le cautele e i dubbi sulla sicurezza, l'azienda ha confermato l'intenzione di procedere con lo sbarco in Borsa (Il Sole 24 Ore - Tecnologia).

* Una discussione su Hacker News solleva interrogativi sul ruolo delle aziende di IA nella definizione delle normative vigenti. In particolare, si chiede una regolamentazione più concreta su temi come il conteggio trasparente dei token di input e output e la qualità effettiva dei servizi (Hacker News).

La Svizzera vota no a rafforzare la neutralità nella Costituzione — Il Sole 24 Ore - Mondo

Kiev, 14 morti e 57 feriti in 24 ore per i raid russi — Il Sole 24 Ore - Mondo

Allarme esplosivi vicino base in Gb utilizzata dagli Usa. Trump: gli arrestati erano sotto indagine — Il Sole 24 Ore - Mondo

I bot di OpenAI violano siti del governo Usa — Il Sole 24 Ore - Mondo

Guerra, ultime notizie. Iran, Trump: mi aspetto nuovi colloqui con Teheran già in settimana — Il Sole 24 Ore - Mondo

Benzina e diesel, anche Ip fisserà limiti ai prezzi — Il Sole 24 Ore - Economia

Vita-lavoro, certificazione al via — Il Sole 24 Ore - Economia

Halloween, è iniziato il conto alla rovescia per la notte dei fantasmi — Il Sole 24 Ore - Economia

«Regate in mezzo al pubblico come mai, Luna Rossa e New Zealand i team favoriti» — Il Sole 24 Ore - Economia

Consolato americano a Milano, la società americana risarcisce i lavoratori con 30 milioni — Il Sole 24 Ore - Economia

Musica, la lotta ai bot che rubano ascolti e compensi agli artisti — Il Sole 24 Ore - Economia

A Napoli alberghi occupati al 100%, ma la provincia resta esclusa — Il Sole 24 Ore - Economia

Napoli, dalle pre regate una forte spinta al rilancio della città — Il Sole 24 Ore - Economia

Truffe e privacy, con l'AI sale il rischio dovuto ai lavoratori sulla sicurezza informatica — Il Sole 24 Ore - Economia

Librino come Versailles, l'ultimo progetto del mecenate Antonio Presti — Il Sole 24 Ore - Economia

L'AI riscrive le regole dei test: Astra aggancia Claude in vetta all'Index — Il Sole 24 Ore - Tecnologia

EA Sports FC 27 cambia la difesa: ora l'intelligenza artificiale aiuta meno — Il Sole 24 Ore - Tecnologia

Il Bimby guadagna l'AI: Cookidoo è l'assistente chef integrato — Il Sole 24 Ore - Tecnologia

OpenAI, sospeso addestramento modelli dopo incidenti con agenti AI. Nonostante i dubbi sbarco in Borsa confermato — Il Sole 24 Ore - Tecnologia

Ask HN: Why do we let the AI companies dominate regulation discussion? — Hacker News (top stories)